

The Action Layer — Universal Hands

How Proxy *does* a task instead of only talking about it — and proves the result before it says a word. Part A is the product: what the hands deliver, across coding, finance, sales, and design. Parts B–D are the build — the one machine every request flows through, tool by tool, exactly what each piece is, how we wire it, and how we optimize it for any task at any size.

Everything else in Proxy is judgment about *what* to do. This is the doing — and the proving. When a person says "run that test," when Proxy notices a claim worth checking, when an expert needs to look something up, or when training probes a unit: all of it bottoms out in one shared capability that builds a result and shows exactly how it knows the result is right.

THE ONE IDEA TO HOLD

There is **one pair of hands** in Proxy, and everyone calls it the same way. A request arrives as a small sealed **envelope** — the task, pointers to its context, who's asking, how fresh the answer must be, and whether it may cause side effects. A ~10-millisecond piece of plain code decides how much effort the task truly needs — most tasks are already *known* and answered in a second — and only the genuinely new work opens the **workshop**, a locked room where one worker plans, builds, and proves before anything comes out. Every result leaves as a second envelope: the artifact, a **receipt** proving how it was checked, and — for anything that touches the outside world — a **staged draft** that a named human must confirm. Same hands for a two-line ticket and a monorepo build; the machine just spends more where the task earns it.

PART A — THE PRODUCT

What the hands do, and why one pair covers everything

The capabilities a product person needs to know — written before any implementation detail.

The action layer delivers a single, remarkable promise: Proxy can **do essentially any task and prove it** — not summarize, not suggest, but actually build the artifact, run the check, and hand back evidence. Four things are always true, no matter the task.

- **It handles any kind of action.** Read a number, run a test, rebuild a model, compose a document, stand up a working calculator, generate a mockup — the same machine handles all of them. New kinds of task need no new engine; they arrive as the same envelope and flow down the same path.
- **It is agnostic to size and vertical.** The identical layer serves a coding meeting, a finance review, a sales call, a design crit, and a board discussion. Nothing in it knows what industry it's in — only the company's estate and the experts in the room differ. A two-line ask and a multi-hour simulation ride the same rails; the machine spends compute in proportion to what the task actually needs.
- **It always ends in proof.** Nothing is ever asserted unproven. A result comes back with a *receipt*: what ran, against which sources, when, and — honestly — what could *not* be proven. If Proxy can't fully verify something, it says so plainly rather than bluffing.
- **Anything that changes the world is a draft first.** Proxy never quietly does something irreversible. A pull request, a posted message, a filed ticket — each comes back as a **staged draft** that a named person confirms with one click. Proxy proposes; a human commits.

What this looks like across four very different meetings

The point of "universal hands" is easiest to feel by watching the *same machine* handle wildly different work. In each case the request enters as one envelope and the workshop runs the same four moves — only the tools and the proof differ.

Coding — reproduce a race-condition bug and stage the fix. Someone says "I think checkout double-charges under load." Proxy's `checkout` expert is already in the room. The hands spin up a private sandbox, write a stress harness that hammers the payment path concurrently, catch the double-charge, write the fix, and run the test again to show it's gone. What comes back is not "you should add a lock" — it's a **staged pull request** with the failing-then-passing test attached as its receipt. If the race proved too flaky to reproduce deterministically, Proxy stages the fix anyway and says so on the receipt: "built the fix; could not deterministically reproduce the race — here's what I ran." Honest, not a false claim.

Finance — rebuild a revenue model with confidence bands. "What does next year look like at 9% churn?" The hands rebuild the revenue model in a sandbox, run a *Monte-Carlo simulation* — thousands of randomized runs that produce a range rather than a single fragile number — and hand back the projection *with bands* (a likely middle and a spread). Because a full simulation rarely fits the two seconds a live answer wants, Proxy speaks the point estimate immediately and lands the full bands in the side panel seconds later — both as a staged draft, both with the exact inputs and assumptions on the receipt.

Sales — build a live ROI calculator during the call. "Show them what they'd save." The hands generate a small working calculator, pre-filled with the customer's own usage numbers where they've been attached to the meeting, and return it as an interactive artifact the account team can drive on screen. Where the customer's real data isn't available, it degrades gracefully to a template with sensible defaults, clearly marked — never a fabricated saving.

Design — generate mockups grounded in the real design system. "Give me three takes on the new settings screen." The hands generate interactive mockup variants built from the company's *own* design tokens and components — the colors, spacing, and buttons that already exist in the codebase — so the output looks like the product, not a generic template. If the design system isn't fully mapped yet, Proxy produces the variants and flags exactly which parts aren't yet bound to real tokens.

WHY ONE MACHINE CAN HONESTLY CLAIM "ANY TASK"

The trick is not a giant library of task-specific skills. It's that every task decomposes into the same four moves — *state what "done" means, plan how to reach it, build it in a locked room, and prove it* — and a general-purpose worker with the right tools can run those moves against anything. When a task has no off-the-shelf way to check it, Proxy **invents the check** as part of planning, has a second model clear that check, and — if it still can't fully prove the result — returns an honest partial receipt rather than a confident lie. That single discipline is what lets one pair of hands span a race bug, a revenue model, an ROI tool, and a mockup without a line of vertical-specific code.

Three product features ride directly on this layer

Because every real job produces a reusable recipe, three user-facing features fall out for free. **Saved Actions:** any completed job can become a named, parameterized, shareable card — "save that as *weekly-metrics-pull*" — invocable by name later ("run *weekly-metrics-pull*, EU only") and eventually schedulable. A team's library of these becomes real switching cost. **Dry-run:** ask "what *would* you do here?" and Proxy returns the plan without executing a thing. **Show-your-work:** ask "where did you get this?" and any result expands to its full step-by-step trace. Saved Actions can be created by a spoken request, inferred from a recurring shape, or gently suggested — and **suggestions never interrupt a meeting**; they wait in the panel or the post-meeting brief.

PART B — HOW IT WORKS

The machine, end to end

The whole flow in one picture, then in words.

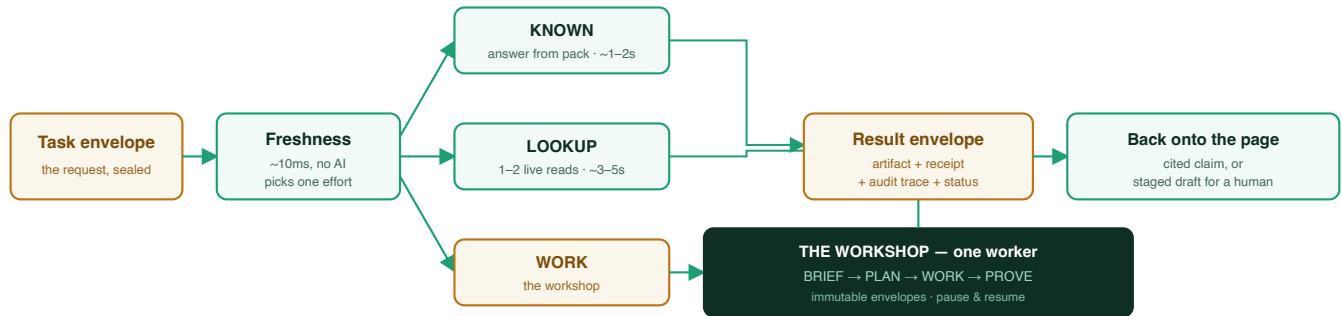


Figure 1 — One request, one path. Every caller hands in a Task envelope; a ~10ms freshness check picks exactly one of three efforts; KNOWN and LOOKUP answer directly, only WORK opens the workshop; everything exits as a Result envelope that flows back onto the meeting page — as a cited claim, or, if it touches the world, as a staged draft.

The flow reads left to right and the same for every caller. A **Task envelope** arrives — a small, sealed object holding the task, *references* to its context (pointers, not fat copies), an identity-and-permissions stamp (who is asking and what they're allowed to touch), a priority and deadline, the stakes, and the crucial *effects* flag: `read-only` or `draft-staging`. The **freshness classifier** — a tiny piece of plain code, no model call — reads how fresh the answer must be and sends the task down exactly one of three efforts. Most tasks are **KNOWN** and answered on the spot. Some are **LOOKUP** and need one quick live read. Only the genuinely new ones are **WORK** and open the **workshop**, where one worker runs four moves — brief, plan, work, prove — behind a locked door. Every path ends in a **Result envelope**: the artifact, a receipt proving how it was checked, the full audit trace, the status, and what it cost in time and money. That envelope flows back onto the meeting page as a new cited fact — or, for anything side-effecting, as a staged draft awaiting a named human's confirmation.

SAME HANDS, DIFFERENT CALLERS

Four different parts of Proxy call these hands, and they all use the identical envelope. A person's spoken ask ("run the load test"). A proactive candidate Proxy generated on its own ("that scaling claim is worth checking"). A knowledge-system training probe from Deep Dive 01. A referent lookup from the live Scribe. To the action layer they are indistinguishable — a task, some context, an identity, a freshness need. That's what makes it *universal*: nothing upstream has to know how the hands work, and the hands never have to know who called.

How a request finds the right expert — routing without a router

Proxy sits in the meeting as one identity with a *team*: the per-unit expert agents from Deep Dive 01, each the standing authority on one slice of the company's software. But it doesn't hold hundreds of them in mind at once. As people talk, the Scribe resolves every referent — "that migration," "checkout," a metric — to its node in the estate graph, and **each resolution wakes the owning expert to the bench**: the live, shifting set of experts following this meeting, each brought in cache-warm and handed the slice of the page that concerns it. When the topic moves on, that expert stands down.

This is the elegant part: **routing a task to the right expert is a graph lookup, not a decision**. There is no classifier, no "which agent should handle this?" model call. The page's own links already say which node a demand belongs to, and the ownership edges say which expert owns that node. Follow the edge — that's the routing, and it's free and instant. A demand no system owns goes to a fully-capable **generalist** (all tools, every expert available as a consultant — a real floor, never a shrug). A demand spanning two systems has the *primary owner lead* while the neighbor expert is *consulted as a tool*, both sharing one context id so nothing is lost in the handoff.

Every piece, exactly — what it is, how we wire it, how we optimize it

For each component: first the overview in plain terms, then the technical specifics — how we integrate it, how we tune it, and how it's configured to serve any task. Dense, but built to be understood.

The two envelopes — the spine everything hangs on

Overview. The whole layer is held together by two small, sealed objects. Everything flows in as a **Task envelope** and out as a **Result envelope**. Making these the single interface is what lets four different callers share one machine and lets a long job pause and resume cleanly.

What's inside, precisely. The Task envelope carries: the task itself; *references* to context (node ids, check-pointer addresses, page slices — pointers, never fat copies, so the envelope stays tiny and the context stays live); an identity/permissions stamp (the requesting user and what they may touch); priority; deadline; stakes; and the `effects` flag, either `read-only` or `draft-staging`. The Result envelope carries: the artifact; the **receipt** (how it was proven — the exact checks, sources, and timestamps, plus what could *not* be proven); the full audit trace; a status; and the cost in tokens, dollars, and wall-clock time.

How we optimize it. Every envelope is **immutable and freshness-stamped** — a stage never edits its input, it emits the next envelope. This is not bookkeeping pedantry; it's what makes a long job *pausable and resumable without re-running tool calls*. If a build spans past the meeting, we resume from the last envelope boundary rather than starting over. Envelopes live as rows in one plain Postgres instance — the same database as the estate graph — so there's no second store to run.

The three efforts — a ~10ms freshness state machine, no model call

Overview. The single most important optimization in the whole layer is this: a tiny piece of deterministic code decides how hard to work, in about ten milliseconds, *before any AI runs*. This is why heavy machinery *structurally cannot* fire on a simple question — the code physically never routes it there.

How the classifier works. Every piece of context Proxy holds carries a *freshness stamp* with two parts: a **type** — `STATIC` (timeless, e.g. "what does this function do"), `VERSIONED` (can be superseded by a newer version), or `EVENT` (true only in a time window) — and a **validity**: valid or

expired, judged against the check-pointer metadata the estate graph already maintains. The classifier reads those two fields and picks one effort. No embeddings, no LLM, no judgment call — just a state-machine lookup.

- **KNOWN** — the answer already exists in a trained expert pack, on the page, or in a prior result, and its freshness is **STATIC** -and-valid. One cached model call. ~**1–2s**. The existing citation is the proof — no criteria, no plan, no workshop. *Most demands end here.*
- **LOOKUP** — the answer is checkable but **VERSIONED** / **EVENT** or expired. One or two live reads through the check-pointer address the expert already carries — a direct fetch (MCP read) against the real system. ~**3–5s**. The retrieval itself is the proof. Often the answer is *already prefetched* by the companion panel's watcher, so it's instant.
- **WORK** — nothing cached answers it; something must be built, run, or composed. The workshop opens. Effort scales inside WORK too: *bounded* (draft a ticket, summarize docs) gets a two-line criteria and a one-shot check in seconds; *real* (a load test, an ROI model) gets the full treatment; *heavy* (a monorepo build, a big simulation) gets bigger compute and always runs asynchronously with a ping on done.

The one rule that makes it safe: ambiguity always rounds UP. Any **VERSIONED** , expired, out-of-window, or missing-check-pointer signal pushes the task to a higher effort. The reasoning is asymmetric: a stale answer spoken confidently in a meeting is the unforgivable failure, while an unnecessary live check costs about three cents. So the classifier is deliberately biased toward doing more, never less. In practice the full criteria→plan→workshop→proof machinery runs only perhaps **3–8 times per meeting** — exactly where its cost is dwarfed by the value of getting it right.

WHY "WHO OWNS CHECKOUT?" NEVER TOUCHES THE WORKSHOP

A question like "who owns this service?" or "what does this function return?" is **STATIC** -and-valid: it's in the expert's trained pack, and it can't have gone stale in a way that matters. The state machine reads that and routes it to KNOWN before a single sandbox thought happens. The workshop — with its planning, its microVM, its proof cascade — is not *declined* for simple queries; it is never *reached*. That structural impossibility, not a cost heuristic, is why Proxy stays cheap and fast on the ninety percent of demands that are simple.

The workshop — four moves of one worker

Overview. When a task is WORK, it enters the workshop under two house rules: *state what "done" means before you start, and nothing leaves without its proof*. Inside, it isn't a relay of separate systems

— it's the reflexes of **one worker** running four moves, connected by immutable, freshness-stamped envelopes so a long job can pause and resume without re-running any tool call.

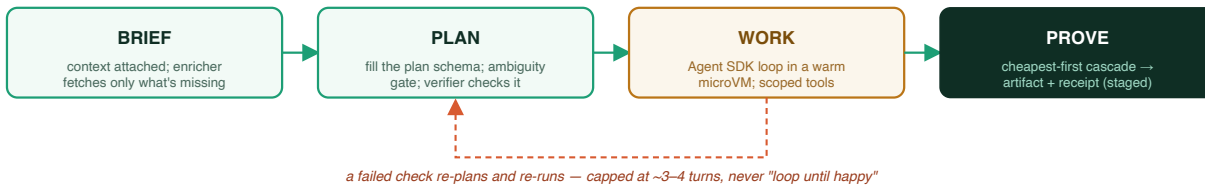


Figure 2 — The workshop's four moves. One worker, connected by immutable envelopes. The orange path is the discipline: a failed proof forces a bounded re-run, hard-capped at three to four turns because the evidence says gains flatten after that.

Move 1 — BRIEF: attach context, fetch only what's missing

What it is. The context is *already* mostly attached — the envelope carried the resolved referents, the check-pointers, the stakes, the deadline, the identity. BRIEF's only job is to notice a gap. If the worker judges the attached context insufficient, a **thin agentic enricher** fetches exactly what's missing — and nothing else.

How we optimize it. We do *not* dump a big static context blob and hope. We run a small "decide-what's-missing-then-fetch" loop (a belief-state pattern: keep a checklist of what the task needs, fetch against the graph or the page only for the unchecked items, stop when the list is satisfied). This is just-in-time context — the difference between a lean brief and a bloated one that slows every downstream call. If a brief ever overflows the model's context window, a compaction step shrinks it; that's a reserve tool, never the default.

Move 2 — PLAN: one strong call into a fixed schema, guarded three ways

What it is. One strong model call composes the plan into a **fixed schema** — a small form with named blanks: `{acceptance-criteria, steps, tools, evidence-needed, stop-conditions, compute-class, live|async}`. "Acceptance criteria" is the heart of it: an explicit statement of what "done and correct" means, written *before* any work, so the worker has something concrete to prove against.

How we make the plan reliable — and why it's not a multi-model debate. Three guards, each cheap:

- **Constrained decoding (XGrammar)** guarantees the plan is schema-valid as *the model generates it* — the model literally cannot emit a malformed plan, rather than us generating freely and validating after. We chose XGrammar specifically because it's fast enough (up to 14× quicker on

JSON, up to $80\times$ on complex grammars than the alternatives) to fit inside the 2–4 second plan budget; slower constrained-decoding libraries that take tens of seconds to compile a schema are disqualified outright.

- **A semantic-entropy ambiguity gate** asks the model to interpret the acceptance criteria a few times in parallel and checks whether it agrees with *itself*. Disagreement means the request is genuinely ambiguous — so Proxy asks one clarifying question or honestly refuses *before* spending a cent on work, rather than confidently building the wrong thing.
- **A deterministic verifier** — plain code, no extra model — checks the finished plan: do all referents resolve? does every criterion map to a step? is the tool list sane? This is the "LLM-modulo" pattern: the model proposes, cheap code disposes. We deliberately reject multi-model plan-refinement loops here — the evidence is clear that they add hops and tokens without beating one strong constrained call.

Where the plan's wording comes from — nothing hand-written. The instruction text for these generation steps isn't authored by an engineer and frozen. It's **compiled nightly from our own logs** by an optimizer (DSPy/MIPROv2): it reads the shadow ledger of past jobs, proposes better instructions, and deploys a new frozen prompt *only if it beats the current one on a held-out regression set*. We cap prompt length and gate every deployment on that regression check because the failure mode is documented and real — over-optimized prompts can bloat and *lose* accuracy. So the plan's phrasing improves month over month, per organization, without a human editing a prompt.

THE PLAN PASS — SCHEMA AND RULES (ILLUSTRATIVE; THE INSTRUCTION TEXT IS DSPY-COMPILED)

Given the task, its attached context, identity, stakes and deadline,
emit ONE plan object. XGrammar constrains you to this schema as you write:

```
{
  acceptance_criteria: [ "testable statement of done", ... ], # what PROVE checks
  steps:              [ "ordered action", ... ],
  tools:              [ "tool name the step needs", ... ],   # tool-search resolves
  evidence_needed:    [ "what the receipt must show", ... ],
  stop_conditions:    [ "when to halt", ... ],
  compute_class:      none | e2b | modal | modal-gpu | self-host,
  mode:               live | async
}
```

For a NOVEL task with no standard verifier: SYNTHESIZE a verification plan
into acceptance_criteria (how success will be proven). A cheap judge model
then clears or rejects that plan BEFORE any work is paid for.

Do not begin work. If the ambiguity gate flags the criteria as undecidable,
ask ONE clarifying question or refuse honestly — never guess.

Move 3 — WORK: the Agent SDK loop, headless in a locked room

What it is. The plan hands off to the actual builder: a **Claude Agent SDK loop** — think → call a tool → read the result → repeat — running *headless* (no screen, fully automated) inside an **E2B microVM**, a tiny throwaway virtual machine (a "microVM" is a lightweight, hardware-isolated computer that boots in a fraction of a second). We keep a **warm pool** of these ready, so a job starts executing near-instantly instead of waiting to boot.

How the worker gets tools — and only the ones it may use. We do not hand-maintain a tool list. The full catalog of connectors lives behind an **MCP registry and gateway**: the registry (Smithery / Docker's catalog) is for discovery and version-pinning; the gateway sits in front and, per job, mints **scoped tokens that live at most 24 hours** and permit only the specific operations this job needs. At runtime, **Anthropic tool-search** lets the worker discover the right 3–5 tools out of thousands and load only their schemas — so every tool is *reachable*, only permitted ones are *usable*, and the prompt never bloats with a giant tool list. For reads with no maintained connector we fall back to browser automation (Browserbase/Stagehand); for a truly custom internal API the worker writes a disposable client in the sandbox itself.

How we keep it safe and bounded. A **PreToolUse hook** — a gate that fires before any tool call lands — lints, typechecks, and dry-runs every mutation before it's allowed into the artifact, so a broken edit never even enters. Sub-agents are allowed but only *one level deep* and only where work is genuinely independent (parallel where it helps, never a deep tree). And iteration is **hard-capped at about 3–4 turns** of run-fix-run, because the measured returns flatten after that — we take the first verified finding and stop, rather than looping until "satisfied."

Move 4 — PROVE: a cheapest-first cascade, then a staged draft

What it is. Nothing leaves the workshop unproven. PROVE runs a **cascade, cheapest check first**, short-circuiting the moment something passes cleanly, so we never pay for expensive proof when cheap proof suffices.

- **Deterministic checks first** (microseconds to milliseconds, no AI): does the artifact match its schema and contract? do property tests hold — every referent resolves, every criterion maps to an output? If a plain-code check settles it, we're done.
- **A grounding ensemble next** (~24ms): a fast HHEM-style verifier checks that every claim in the result is actually supported by its sources — catching a statement that *looks* grounded but drifted.
- **An agent-as-judge last, only for high stakes**: a verifier that *re-derives the claim with real tools* rather than just reading the text — the only thing that catches a result that reads plausibly but is

factually wrong. It's the most expensive tier, so it's reserved for high-stakes claims, not run on every artifact.

The result is packaged as **artifact + receipt** — what ran, which sources, timestamps, and honestly what could *not* be proven. And the keystone: anything side-effecting exits **only as a staged draft** that a named human confirms. Proxy proposes; a person commits.

Handling a task nobody has a verifier for

The hard case, honestly. Most tasks have an obvious check — run the test, compare the number. But some genuinely novel task has *no* standard way to prove it. Here the PLAN pass **synthesizes a verification plan** as part of the acceptance criteria ("here is how I will demonstrate this worked"), and a cheap judge model clears that plan before work begins — because a self-authored check is inherently weaker than the work it checks and needs a second eye. Then, critically, PROVE **degrades gracefully**: if it built the thing but genuinely couldn't fully verify it, it returns an *honest partial receipt* — "built it; proved X; could not prove Y, here's exactly what I ran" — never a false claim and never a hard block. This matters because verification of a novel codebase fails outright roughly a third of the time even with retries; the honest partial receipt is what turns that ceiling into a trustworthy behavior instead of a lie or a dead end.

Configuring the workshop for any use case

Overview. The reason this one workshop covers coding, finance, sales, and design is that *almost nothing in it is task-specific*. What differs per task is data, not code.

What actually varies. Only three things change from a race-bug task to a Monte-Carlo model to a mockup: (1) the acceptance criteria the PLAN pass synthesizes for *that* task; (2) the 3–5 tools tool-search retrieves for it; and (3) the compute rung the plan picks. The worker, the sandbox, the proof cascade, the safety stack, the envelopes — all identical. A new class of task or a new connector is added by **registration, not engineering**: drop the connector in the registry, and both a reactive ask and a proactive candidate can exercise it with zero engine change. That is what "size- and vertical-agnostic" means concretely — there is no branch in the code that knows what industry it's in.

The safety stack — stated honestly, including what it can't do

Overview. The workshop runs code influenced by untrusted meeting content, holding real tools. We defend it in layers, and we're honest about the one risk no one can zero out in 2026.

- **Firecracker microVM, VMM unprivileged + seccomp** — the isolation floor. The virtual-machine monitor runs without privileges and with a syscall filter, which defeats hostile generated

code trying to escape the sandbox. This is a *tested invariant*, never assumed: a pre-flight assertion refuses to launch a privileged VMM, because running it as root would collapse the floor.

- **Egress allowlist, default-deny** — per job, the sandbox may talk only to explicitly permitted hosts. This is the single strongest control against data exfiltration once the sandbox has network access.
- **No live secrets in the sandbox** — the real credentials never enter. The gateway (via Nango) mints scoped, short-lived proxy tokens per job; the sandbox holds a limited stand-in, never the actual key.
- **Capability confinement + taint tracking** — untrusted meeting content can influence *what* is asked, but is stripped of the authority to *dispatch* a sensitive tool; data derived from untrusted input is tainted and gates sensitive calls.
- **Staged-drafts-only side effects — the keystone.** Every action is a proposed draft a named human confirms. This is the load-bearing safety property of the entire product: it converts "injection causes an action" into "injection causes a *rejected proposal*." Guard it absolutely.
- **Full tool-call telemetry** — every prompt, tool confirmation, result, and network allow/deny is logged, giving us an audit trail and a red-team substrate.
- **Risk-scaled autonomy** — low autonomy for reads and reports, higher only for staged pushes; autonomy matches the task's risk.

The residual risk, stated plainly. A prompt-injected agent can still surface *subtly-wrong information* within its allowed reads — it read only what it was permitted to read, but an injection steered *which* facts it surfaced. The draft gate catches bad *actions*; it does not catch bad *information*, and that cannot be driven to zero today. The honest buyer line: escape-proof to enterprise standard; injection-*resistant* by design with human-confirmed drafts as the backstop; not injection-*proof* — nobody's is in 2026, and we show you exactly where the line is.

The learned layer — only schemas, floors, and gates are fixed

Overview. Nothing semantic in this layer is hand-written and frozen. The *only* fixed things are the plan schema, the deterministic verifiers, the ambiguity-gate threshold, the tool-permission scopes, the regression gate, and the latency/cost budgets. Everything else is either compiled from our own logs or generated fresh at runtime.

What's learned vs generated. *Learned* (DSPy-compiled offline, nightly, from the shadow ledger, deployed only past a held-out regression gate): the intake-enrichment prompt, the criteria-synthesis prompt, the validation-rubric prompt. *Generated at runtime* by the agent, per job: the brief (the enricher's belief-state loop), the plan instance, the tool selection (tool-search), and which claims

escalate to the agent-as-judge. The compilation runs on the cold path (Inngest, nightly) reading Postgres logs — never on the hot path, so it can't add a millisecond to a live answer.

How the whole layer stays fast, cheap, and durable at any size

The compute ladder — routed at plan time

The plan's `compute-class` field routes a job to the right rung *before* spin-up, so we never over-provision. **No sandbox** for KNOWN/LOOKUP (they never touch one). **E2B/Firecracker** is the default for most WORK — code edits, spreadsheets, mockups, calculators — booting in a fraction of a second from the warm pool. **Modal or Daytona** for heavy CPU/RAM or long sessions — monorepo builds, big Monte-Carlo sims, large data. **Modal-GPU** for ML inference. And a **Fly / self-host** escape hatch for cost control at extreme scale. Default is E2B; escalation is a plan-time decision, never a runtime scramble.

The optimizations, in one place

- **Heavy machinery can't run on simple queries.** The tier structure isn't a cost preference — it's structural. The ~10ms classifier routes simple demands to KNOWN before any workshop code is reachable, so the expensive path is *unreachable* for the ninety percent of demands that are simple.
- **Warm pools make execution feel instant.** A pool of ready microVMs means a WORK job starts building immediately rather than waiting to boot a machine.
- **Cheapest-first proof.** PROVE short-circuits on the first clean deterministic check; the expensive agent-as-judge fires only for high-stakes claims.
- **Single strong run plus a gate — not best-of-N.** The evidence is clear that best-of-N voting and critic re-ranking *lose* to one strong run with a verification gate. We default to the cheaper, better option and reserve any voting for a narrow high-stakes class only if it measurably helps.
- **Hard iteration cap (~3–4 turns).** Run-fix-run stops where the measured gains flatten, not when the model feels done.
- **Pre-spend cost circuit-breakers.** Each expensive step checks a per-job token/dollar cap *before* spending, plus an org ceiling and a kill switch — the documented failure mode is an unbounded retry loop burning tens of thousands of dollars, and pre-spend caps are the load-bearing defense against it.

- **Tool-result caching with TTL.** Idempotent reads are cached (keyed on the tool and a hash of its arguments) so a repeated lookup is free until its time-to-live expires.
- **Durability without a heavy orchestrator.** The hot path is a plain Postgres `jobs` table plus Agent SDK session resume — nothing more. Only the cold, long, cross-meeting path (a job that outlives the meeting, a human-confirmation pause) uses a durable-execution engine (Inngest). There is deliberately **no Temporal, LangGraph, or CrewAI on the hot path**: three research passes agree that orchestration hops cost latency, multi-agent loses on our shape, and durable execution earns its place only where a named constraint forces it — which is the cold path.

Done when

This layer is complete when: the ~10ms freshness classifier routes every demand to KNOWN/LOOKUP/WORK with no model call, and simple demands provably never reach the workshop; the bench tracks the conversation — an expert wakes when its system is referenced and stands down when the topic moves — and routing to the right expert is a graph lookup needing no classifier or LLM; a WORK job synthesizes its acceptance criteria, clears the ambiguity gate (or asks/refuses), executes in the sandbox, and returns artifact + receipt with nothing unproven asserted; a novel task with no standard verifier degrades gracefully to an honest partial receipt rather than a block or a false claim; every side-effecting result is a staged draft chained to a named human's confirmation in the audit log; the compute ladder escalates by plan-time class and heavy jobs run async with a completion ping; DSPy-compiled prompts deploy only past a held-out regression gate, with zero hand-authored runtime prompts and only schemas, floors, and gates fixed; the safety invariants (unprivileged VMM, egress allowlist) are enforced as tested pre-flight assertions; a new task class or connector is added by registration and exercisable by both a reactive ask and a proactive candidate with no engine change; Saved Actions can be created or suggested without ever interrupting a meeting; and the identical layer serves coding, financial, sales, design, and board meetings with only the estate and the bench differing.